

ProdScheduler v4 — Programación de Producción

Sistema de programación de producción para 2 líneas (Comedores y Sofás) con CRUD inline, Gantt interactivo, calendarios por turnos, metas diarias y asistente de IA.

Uso local (sin instalación)

Descarga `scheduler.html` → doble clic → se abre en cualquier navegador moderno. **No requiere Node.js, npm, ni servidor.** React y Babel se cargan desde CDN.

Tus datos se guardan automáticamente en el navegador (localStorage). Cierra la pestaña y al volver, todo sigue ahí.

Publicar online en GitHub Pages

Una vez publicado, cualquier persona accede vía URL pública (`https://tu-usuario.github.io/scheduler`) sin instalar nada.

Paso 1 — Crear el repo en GitHub

1. Ve a github.com/new
2. Repository name: `scheduler` (o el nombre que quieras)
3. **Public** (Pages no funciona en repos privados con cuenta gratuita)
4. Marca "Add a README file"
5. Click **Create repository**

Paso 2 — Subir el HTML

Opción A (web, sin git):

1. En tu repo, click **Add file → Upload files**
2. Arrastra `scheduler.html` y `README.md`
3. **IMPORTANTE:** GitHub Pages busca `index.html` por defecto, así que renombra `scheduler.html` → `index.html` antes de subirlo (o agrega un `index.html` que redirija a `scheduler.html`)
4. Commit message: "Initial deploy"
5. Click **Commit changes**

Opción B (línea de comandos):

```
bash
```

```
git clone https://github.com/tu-usuario/scheduler.git
cd scheduler
cp /ruta/a/scheduler.html index.html
git add .
git commit -m "Initial deploy"
git push
```

Paso 3 — Activar GitHub Pages

1. En tu repo → **Settings** → **Pages** (menú izquierdo)
2. **Source:** Deploy from a branch
3. **Branch:** `main` / `(root)`
4. **Save**
5. Espera 1-2 minutos. GitHub te dará la URL pública: `https://tu-usuario.github.io/scheduler/`

Paso 4 — Actualizaciones futuras

Cualquier cambio que pushees a `main` se publica automáticamente en 30-60 segundos. Para editar:

```
bash

git pull
# editas index.html
git add . && git commit -m "Update" && git push
```

Persistencia de datos — Tres modelos

La app guarda automáticamente en **localStorage** del navegador del usuario. Esto significa:

Lo que SÍ persiste

-  Todas las órdenes que captures
-  Cambios en máquinas, SKUs, etapas, turnos
-  Metas diarias y matriz de setup
-  Secuencias prohibidas
-  Reordenamientos manuales (drag-and-drop)

Lo que NO persiste

-  **Datos compartidos entre usuarios.** Cada persona que entra a la URL tiene SU PROPIA

copia. Si tú agregas una orden, otros usuarios no la verán.

- **✗ Datos entre dispositivos.** Tu laptop y tu celular son "usuarios" distintos para localStorage.
- **✗ Datos al limpiar caché o usar modo incógnito.**

¿Necesitas que varios usuarios vean los mismos datos?

Te quedan tres opciones, de más simple a más compleja:

1. Compartir snapshot por JSON (incluido) Usa los botones ↓ **Exportar** y ↑ **Importar** en la barra superior. Envía el archivo JSON por email/Slack, y el receptor lo importa. Workflow manual pero gratuito.

2. Backend serverless (Firebase / Supabase) Para sincronización en tiempo real:

- Crea cuenta en supabase.com (tier gratuito)
- Crea una tabla `scheduler_data` con columnas `id` (uuid) y `data` (jsonb)
- Reemplaza las llamadas a `localStorage.setItem/getItem` por `supabase.from('scheduler_data').upsert()`
- ~30 líneas de código adicionales

3. GitHub como base de datos (gratis pero hacky)

- Crea un Personal Access Token en GitHub con permiso `repo`
- Usa la API de Gist o Contents para guardar el JSON
- Cualquiera con el token tiene write access
- Funciona, no es elegante, pero \$0 de costo

Si quieres, puedo guiarte con la opción 2 (Supabase) que es la más limpia.

Asistente de IA (opcional)

El asistente requiere una API key de [Anthropic](https://anthropic.com). Sin ella, todas las demás funciones siguen trabajando — solo el chat queda deshabilitado.

Cómo configurar (si decides usarlo): La integración actualmente está hardcodeda con un endpoint placeholder. Para hacerlo funcionar en producción tienes dos rutas:

- **Proxy serverless** (recomendado): un endpoint que reciba el mensaje y llame a Anthropic con tu API key del lado servidor. La key nunca toca el navegador.
- **API key en `localStorage`**: añade un campo de configuración donde el usuario pegue su propia key. Cada usuario usa la suya.

No metas API keys en el HTML que subes a GitHub. Si lo haces, las keys quedan públicas para siempre, incluso si después las borras (el historial git las preserva).

📁 Archivos

Archivo	Descripción
<code>scheduler.html</code>	App completa, todo-en-uno. Renómbralo a <code>index.html</code> para Pages
<code>production-scheduler-v4.jsx</code>	Mismo código pero como módulo JSX (para si quieres usarlo con Vite/Webpack)
<code>README.md</code>	Este archivo

🔧 Stack

- React 18 (vía CDN unpkg)
- Babel Standalone (transpila JSX en runtime)
- localStorage (persistencia)
- Sin frameworks de UI, sin libs externas, sin build step

⚠️ Limitaciones del HTML standalone

- **Carga inicial lenta** (~2s) porque Babel transpila al cargar. En desarrollo está bien; para producción seria, usa Vite + el `.jsx`.
- **Sin code splitting** — todo el código carga junto (97 KB sin gzip).
- **Sin tree shaking** — el bundle es lo que escribiste.

Para una versión optimizada, migra a Vite:

```
bash

npm create vite@latest scheduler -- --template react
# pega production-scheduler-v4.jsx en src/App.jsx
npm install && npm run dev
```